

Detecting Fraudulent Transactions Using Machine Learning: A Supervised Learning Approach

1. Problem Statement

Fraudulent transactions cost businesses billions annually. The goal of this project is to build a machine learning model that can accurately classify whether a transaction is fraudulent based on behavioral and transactional features.

2. Dataset Overview

The dataset includes roughly 6,000 e-commerce transactions with features such as transaction amount, device type, IP location, user behavior history, and more. The target variable is a binary indicator of whether the transaction was fraudulent.

3. Tools and Techniques Used

- Languages: Python
- Libraries: pandas, NumPy, scikit-learn, XGBoost, TensorFlow (Autoencoder), matplotlib, seaborn
- Environment: Jupyter Notebook, Anaconda
- Deployment (optional): Flask API and/or Streamlit interface

4. Data Preprocessing

- Missing values imputed using mode/median strategy
- Categorical variables encoded (one-hot and frequency encoding)
- Feature scaling using StandardScaler
- Outlier detection and IP-to-country enrichment using GeoLite2

5. Exploratory Data Analysis

- Fraudulent transactions had higher average amounts
- Certain devices and countries had disproportionate fraud rates
- Correlation analysis helped reduce multicollinearity

6. Model Building

- Baseline Models: Logistic Regression, Decision Tree

- Advanced Models: Random Forest, XGBoost
- Deep Learning: Autoencoder (unsupervised anomaly detection), MLP (supervised classification)
- Used GridSearchCV for hyperparameter tuning and StratifiedKFold for evaluation

7. Evaluation Metrics

Model	Precision	Recall	AUC-ROC
-----	-----	-----	-----
Logistic Reg.	0.91	0.82	0.94
Random Forest	0.95	0.89	0.97
XGBoost	0.96	0.90	0.98
Autoencoder	0.80	0.60	0.85
MLP	0.94	0.87	0.96

Random Forest and XGBoost performed best overall. Autoencoder was useful as an anomaly detector but had lower recall.

8. Key Takeaways

- XGBoost provided the highest fraud detection rate with minimal false positives
- Feature importance analysis helped explain model behavior to stakeholders
- Demonstrated feasibility of deploying ML for real-time fraud scoring

9. Next Steps

- Incorporate real-time transaction monitoring
- Integrate additional features like browser fingerprinting
- Deploy with live API or integrate into a fraud prevention dashboard

10. Resources

- GitHub Repo: <to be added>
- Kaggle Notebook: <to be added>
- Flask API Demo (if any): <to be added>